



A.表达式的相互转换方法：基于栈的应用

表达式的转换是数据结构中**栈（Stack）**应用的经典案例。转换的目的是消除中缀表达式中括号和优先级的概念，使其能被计算机高效地处理（如计算或编译）。

表达式类型	形式	运算符位置	转换工具
中缀 (Infix)	$A + B$	运算符在操作数之间	需使用操作符栈处理优先级和括号
后缀 (Postfix/RPN)	$AB+$	运算符在操作数之后	适用于栈式计算
前缀 (Prefix)	$+AB$	运算符在操作数之前	适用于栈式计算

1. 中缀表达式 → 后缀表达式 (Infix → Postfix/RPN)

方法：使用一个**操作符栈**来处理运算符的优先级和括号。

✦ 转换规则

- 操作数**：直接输出到后缀表达式。
- 运算符 (θ)**：
 - 当栈顶运算符的优先级**大于或等于** θ 时，弹出栈顶运算符并输出。
 - 重复此操作，直到栈空或栈顶优先级小于 θ ，然后将 θ **压入栈**。
- 括号**：
 - 左括号 '('**：直接压入栈。
 - 右括号 ')'**：持续弹出并输出栈顶运算符，直到遇到 '(' 为止。将 '(' 和 ')' 都丢弃。
- 扫描结束**：将栈中剩余的所有运算符依次弹出并输出。

🎨 示例： $(A + B) \times C - D \rightarrow AB + C \times D -$

2. 后缀表达式 → 中缀表达式 (Postfix/RPN → Infix)

方法：使用一个**子表达式栈**，用于存储子表达式字符串。

✚ 转换规则

1. **操作数：** 将操作数作为一个独立的子表达式**压入栈**。
2. **运算符 (θ):**
 - 从栈中弹出右子表达式 B (先弹出)。
 - 从栈中弹出左子表达式 A (后弹出)。
 - **组合：** 将它们组合成新的中缀表达式： $((A) \theta (B))$ 。
 - **注意：** 组合时必须在 $A \theta B$ 的外部加上**括号**，以保证运算顺序。
 - 将 $((A) \theta (B))$ 压回栈中。
3. **结束：** 栈中剩下的唯一字符串即为中缀表达式 (最外层括号通常可省略)。

 **示例：** $AB + C \times D - \rightarrow ((A + B) \times C - D)$

3. 中缀表达式 $\rightarrow$ 前缀表达式 (Infix $\rightarrow$ Prefix)

方法： 利用**逆序转换**的思想。

1. **逆序中缀表达式：** 将中缀表达式逆序，并交换所有的 '(' 和 ')'。
2. **执行中缀 $\rightarrow$ 后缀转换：** 对步骤 1 的结果，应用中缀 $\rightarrow$ 后缀的转换规则。
3. **逆序结果：** 将步骤 2 得到的后缀表达式结果**再次逆序**，即得到最终的前缀表达式。

 **示例：** $A + B \times C \rightarrow +A \times BC$

4. 后缀表达式 $\rightarrow$ 前缀表达式 (Postfix $\rightarrow$ Prefix)

方法： 使用一个**操作数栈**，存储**前缀子表达式**。

1. **操作数：** 压入栈。
2. **运算符 (θ):**
 - 弹出右操作数/表达式 B (先弹出)。
 - 弹出左操作数/表达式 A (后弹出)。
 - **组合：** 组合成新的前缀表达式 θAB ，压回栈。

 **示例：** $ABC \times +D - \rightarrow - + A \times BCD$

B.三种表达式的定义和形式

这三种表达式形式主要区别在于**运算符（Operator）相对于其操作数（Operand）**的位置。

表达式类型	形式名称	运算符位置	结构示例	核心特点
中缀表达式	Infix Notation	操作数之间	$A + B$ 或 $(A + B) \times C$	易于人类阅读，但计算时需要考虑 优先级和括号 。
后缀表达式	Postfix Notation（逆波兰表达式 RPN）	操作数之后	$AB+$ 或 $AB + C \times$	消除了括号和优先级，可以直接通过 栈（Stack） 进行计算。
前缀表达式	Prefix Notation（波兰表达式 PN）	操作数之前	$+AB$ 或 $\times + ABC$	消除了括号和优先级，也可以通过栈进行计算（通常从右向左扫描）。

 示例对比： $(A + B) \times C$

表达式类型	形式
中缀	$(A + B) \times C$
后缀	$AB + C \times$
前缀	$\times + ABC$

C.表达式的二叉树表示法（表达式树）

表达式树（Expression Tree）是一种将数学表达式表示为二叉树的方法，它能够清晰地体现运算的逻辑和顺序。

构造方法

节点类型	内容	作用
内部节点	运算符（ $+$, $-$, $\times$, $/$, ...）	代表一个运算操作。
叶子节点	操作数（数字、变量）	代表参与运算的对象。

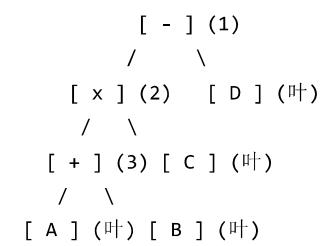
节点类型	内容	作用
左右子树	左、右操作数或子表达式。	树的深度决定了运算的优先级（越深越先算）。

示例：表达式树的构造与遍历

目标中缀表达式： $(A + B) \times C - D$

1. 表达式树结构图（符号绘制）

主运算符（最后执行）是 $-$ ，作为根节点。



(数字 (1), (2), (3) 仅为标记运算的执行顺序)

2. 表达式与遍历关系

通过对这棵树进行不同顺序的遍历，可以直接得到三种表达式形式。

遍历方式	对应表达式	遍历顺序（根、左、右）	结果
前序遍历	前缀表达式	根 $\rightarrow$ 左 $\rightarrow$ 右	$- \times + ABCD$
中序遍历	中缀表达式	左 $\rightarrow$ 根 $\rightarrow$ 右	$A + B \times C - D$
后序遍历	后缀表达式	左 $\rightarrow$ 右 $\rightarrow$ 根	$AB + C \times D -$